

Wave Function Collapse and how design
constraints affect map diversity

Adele Bauer

BSc (Hons) Computer Game Applications
Development, 2026

School of Design and Informatics
Abertay University

Table of Contents

Table of Figures	iii
Acknowledgements	v
Abstract.....	vi
Abbreviations, Symbols and Notation	vii
Chapter 1 Introduction	1
1.1 - Project Overview	1
1.2 - Project Aim.....	3
1.3 - Research Question	3
1.4 - Objectives	4
1.5 - Hypothesis	4
1.6 - Dissertation Overview	4
Chapter 2 Literature Review	5
2.1 – Bad North	5
2.2 – Townscaper	6
2.3 – Design-Level Constraints	7
Chapter 3 Methodology.....	9
3.1 - Overview	9
3.2 - Preparation.....	10
3.3 – Application Design.....	11
3.3.1 – Design of Cells and Ports	12
3.3.2 – Design of the Map	15
3.3.3 – Design of the Algorithm	18
3.3.4 – Design of the Coordinate Constraint	22
3.3.5 – Design of the Survey	25
Chapter 4 Results	31
4.1 – Survey Results from Section 1	31
4.2 – Survey Results from Section 2	38

Chapter 5 Discussion	47
5.1 – Results Discussion	47
5.1.1 – Section 1 Interpretation	47
5.1.2 – Section 2 Interpretation	48
5.1.3 – Overall Survey Interpretation	49
5.2 – Project Discussion	49
5.2.1 – Scope and Motivation	50
5.2.2 – Flawed Survey Design.....	50
Chapter 6 Conclusion and Future Work	51
List of References	52
Appendices	53
Appendix A – Survey Questions.....	53

Table of Figures

Figure 2-1 A Screenshot of Bad North's procedurally generated levels	6
Figure 2-2 A Screenshot of Townscaper's dynamic visual style using Wave Function Collapse	7
Figure 3-1 An Illustration of a straight road cell, with each edge's port displayed.....	12
Figure 3-2 Two Illustrations of 4 identical cells being placed next to each other. The first example fulfils the rules, because the ports that are connected are the same. The second example does not, because road and grass ports are not allowed to connect in this example.....	13
Figure 3-3 A screenshot of a unity component, In this example, the XTunnel cell can either show up anywhere where X is equal to -3, OR it can show up anywhere where X is more than 0, AND Y is over 2.	24
Figure 3-4 A map generated without constraints. Note that two opposing edges can connect to each other, making a thin stream of water	26
Figure 3-5 A map generated with the "WaterEdge" ruleset.....	26
Figure 3-6 A map generated with the "LeftRocks" ruleset.....	27
Figure 3-7 A map generated with the "NoOuterEdge" ruleset.....	28
Figure 3-8 A map generated with the "NoInnerEdge" ruleset.....	28
Figure 4-1 Question 1 of Section 1 of the Survey	31
Figure 4-2 Why / Why Not? for Question 1 of Section 1 of the Survey ...	31
Figure 4-3 Question 2 of Section 1 of the Survey	32
Figure 4-4 Why / Why Not? for Question 2 of Section 1 of the Survey ...	32
Figure 4-5 Question 3 of Section 1 of the Survey	33
Figure 4-6 Why / Why Not? for Question 3 of Section 1 of the Survey ...	33
Figure 4-7 Question 4 of Section 1 of the Survey	34
Figure 4-8 Why / Why Not? for Question 4 of Section 1 of the Survey ...	34
Figure 4-9 Question 5 of Section 1 of the Survey	35
Figure 4-10 Why / Why Not? for Question 5 of Section 1 of the Survey .	35
Figure 4-11 Question 6 of Section 1 of the Survey	36
Figure 4-12 Why / Why Not? for Question 6 of Section 1 of the Survey .	36
Figure 4-13 Question 7 of Section 1 of the Survey	37
Figure 4-14 Why / Why Not? for Question 7 of Section 1 of the Survey .	37

Figure 4-15 Question 1 of Section 2 of the Survey	38
Figure 4-16 Why / Why Not? for Question 1 of Section 2 of the Survey .	38
Figure 4-17 Question 2 of Section 2 of the Survey	39
Figure 4-18 Why / Why Not? for Question 2 of Section 2 of the Survey .	39
Figure 4-19 Question 3 of Section 2 of the Survey	40
Figure 4-20 Why / Why Not? for Question 3 of Section 2 of the Survey .	40
Figure 4-21 Question 4 of Section 2 of the Survey	41
Figure 4-22 Why / Why Not? for Question 4 of Section 2 of the Survey .	41
Figure 4-23 Question 5 of Section 2 of the Survey	42
Figure 4-24 Why / Why Not? for Question 5 of Section 2 of the Survey .	42
Figure 4-25 Question 6 of Section 2 of the Survey	43
Figure 4-26 Why / Why Not? for Question 6 of Section 2 of the Survey .	43
Figure 4-27 Question 7 of Section 2 of the Survey	44
Figure 4-28 Why / Why Not? for Question 7 of Section 2 of the Survey .	44
Figure 4-29 Question 8 of Section 2 of the Survey	45
Figure 4-30 Why / Why Not? for Question 8 of Section 2 of the Survey .	45
Figure 4-31 Question 9 of Section 2 of the Survey	46
Figure 4-32 Why / Why Not? for Question 9 of Section 2 of the Survey .	46

Acknowledgements

Thank you to my supervisor, Thomas Howson, for his advice and help throughout this project.

Thank you to everyone who participated in the survey, your time and effort is greatly appreciated.

Thank you to my classmates who I could go to for advice regarding the module.

Thank you to all my friends, who I have relied on for staying focused and motivated, as well as being there for me when I need a break from university work to socialise.

Thank you to my partner Rebecca, who has given me continuous support and encouragement throughout this project.

This project has been tough due to burnout, but it has also been an amazing experience that I will remember for a long time!

Abstract

In the context of game development, procedural generation is a method of generating content for games. Since this content is generated by an algorithm, it can produce a significantly larger amount of content compared to manually produced content. The most well-known game of all time, *Minecraft* (Mojang, 2009) utilises Perlin noise to procedurally generate its near infinite worlds.

Wave Function Collapse (WFC) is a type of procedural generation, that uses pre-built “cells” to fill up a space, using rules to determine which cells can be placed next to which. The algorithm has been used in some well-known games, such as *Bad North* (Plausible Concept, 2018) and *Townscaper* (Stålberg, 2021).

The project implements design constraints into the WFC algorithm, to further customise the procedurally generated levels. This, however, may potentially reduce the observed level diversity. The project’s goal is to use a survey to determine to what extent this is true, to which it was mostly successful in proving the hypothesis to be correct.

The project’s implementation of WFC was non-standard, but highly effective at simplifying the process of adjusting the generated levels for a potential level designer. As a result, it is possible that an intellectual advancement in the field of Wave Function Collapse has been made, in the case that it proves helpful to future developers of the algorithm.

Abbreviations, Symbols and Notation

ABBREVIATIONS

Wave Function Collapse - WFC

NOTATION

$O(X)$ – Big-O Notation, for describing performance for code or algorithms.

Chapter 1 Introduction

1.1 - Project Overview

Procedural Generation is a common method of creating content in games, by using algorithms or rules to determine what to create and where. The alternative, being manually creating content, can be time consuming and in some cases unnecessary.

A common use for procedural generation is terrain, levels or maps, as these are often large and expansive, and having a procedurally generated map allows for near infinitely large worlds. One of the most famous examples of procedural generation is Minecraft (Mojang, 2009), where all the terrain is generated by an algorithm, and as a result, no two people will encounter the same world seed.

The big reason many games opt to use procedural generation over manually creating maps, is time. Manually creating maps requires a level designer to place every tree, building or asset into the world, and in many cases that amount of work can be the difference between a shipped game and an unfinished one.

Another reason for using procedural generation is to make the game more replayable. Navigating through the same map multiple times can get boring and monotonous, so being able to generate maps at runtime allows for players to always have new content to discover. Manually creating that much content is an unfeasible amount of work for any team, regardless of size, so procedural generation is one of the only options.

Procedural generation does not come without any downsides, however. When a game has a manually created map, it can add a lot of depth to the feel of the game, and that attention to detail can be lost when resorting to procedural generation. Because of this, it is mostly used

when the level design isn't as important as other aspects of the game, such as sandbox games.

Minecraft (Mojang, 2009) is known for having expansive landscapes that are different for every person who plays the game. The world generation uses a gradient noise algorithm, such as Perlin noise, to ensure randomness while still having continuity on a smaller scale. Minecraft uses multiple noise functions with different frequencies and amplitudes and are added together. This method of procedural generation is very cost effective and can be scaled up to the large worlds that are present in Minecraft.

One of the less common methods of procedurally generating maps or levels, is Wave Function Collapse (WFC). This method uses rooms or cells that are manually created once, then utilizes rulesets to decide which cells can be placed next to which and generates a full map where all cells follow the rules given. This method can be computationally heavy for device memory. This is due to repeated calculations that need to be done at every step of generation, and the workload increases exponentially with map size. As a result, it is preferred when generating smaller maps, rather than large landscapes.

Wave Function Collapse takes some of the benefits of procedural generation and combines it with the attention to detail only found in manually created levels. However, it is still lacking in terms of level design principles found in modern games.

Minecraft (Mojang, 2009) handles this by implementing mechanics to allow players to find necessary structures for game progression, such as the ender eye that leads to the nearest stronghold, or ruined portal structures that hint at how to make a nether portal.

By implementing constraints into the algorithm, some design principles can be implemented, such as ensuring some cells only appear in specific places, even if they are compatible with the surrounding cells. These

design implementations are then present in all generated levels, ensuring a consistent experience despite the levels being procedural.

Implementing design principles into procedurally generated levels, especially those from Wave Function Collapse, limits the number of possible levels that can be created, because constraints are inherently subtractive. WFC needs to have a set size decided when generating the level, so the worlds generated from it are finite. It is currently unclear how much adding design constraints can affect the diversity of generated levels however, as diversity is a subjective measurement that varies between individuals.

A common consensus can be found for good examples of diverse levels made from WFC. *Bad North* (Plausible Concept, 2018) is able to generate many unique and diverse levels, by combining a couple different strategies, including having a very large number of cells, and using techniques explained later to hide the fact that the world is on a grid.

1.2 - Project Aim

The aim of the project is to make a Wave Function Collapse algorithm, and implement design constraints into it, and finally test various degrees of design constraints to see how much it affects the level diversity of the generated levels.

The design constraints implemented are a coordinate constraint, allowing specific cells to be limited to specific coordinates on the two-dimensional map, as well as cell weights, that influence the likelihood of certain cells from being used when there is more than one option.

1.3 - Research Question

The Research Question of this project is “Using Wave Function Collapse, how much can design constraints affect level diversity?”

1.4 - Objectives

The project's objectives are:

- To investigate some games that utilize Wave Function Collapse and their implementation of the algorithm.
- To implement these ideas into a custom Wave Function Collapse algorithm, and to add design constraints to allow for more customization.
- To test various design constraints, and asking willing participants to group generated levels together
- To analyse the findings and determine to what extent design constraints affect level diversity.

1.5 - Hypothesis

The project artefact will suggest that level diversity is not compromised by adding design constraints, while keeping in mind that if design constraints are continuously added, eventually the generated levels will not be able to deviate from each other.

Some aspects of the project artefact are expected to be unpolished, as the Wave Function Collapse algorithm is complex and time consuming to implement, and focusing on polish or smaller details will not be a priority.

1.6 - Dissertation Overview

Following this Introduction will be a Literature Review that analyses current games that utilize Wave Function Collapse and discusses how those ideas benefit the design and level design for those games. After this, the Methodology segment will dissect how the project artefact was implemented, as well as how the survey will function. The results of the survey will be presented in the Results section, and those results, as well as the project as a whole, will be analysed and discussed in the Discussion section. Finally, the Conclusion will summarize the project and its findings.

Chapter 2 Literature Review

To be able to effectively implement a working Wave Function Collapse algorithm, significant research into WFC and games that utilise it must be done.

This chapter discusses Wave Function Collapse and how it was used in various existing games. It also discusses some other implementations of the algorithm.

The games *Bad North* (Plausible Concept, 2018) and *Townscaper* (Stålberg, 2021) are two notable games that utilize Wave Function Collapse in different ways.

2.1 – Bad North

Bad North (Plausible Concept, 2018) is a real time, tactical strategy roguelike game, that has the player manage Vikings who are defending various islands from raiders. Each island is procedurally generated using Wave Function Collapse, and it is one of the most widely known examples of WFC being used.

The Wave Function Collapse algorithm in *Bad North* utilises a three-dimensional grid, and a very large selection of cells to use when generating a level. Additionally, it uses tiles that take up more than one slot on the grid, a method named Non-Uniform Tiles WFC, or nut-WFC. All of these aspects come together to make very diverse maps, where it is hard to tell it was even made in a grid at all.



Figure 2-1 A Screenshot of Bad North's procedurally generated levels

Nut-WFC was considered for this project, but it was ultimately decided to not be necessary to answer the research question. It would add more to the scope of the project, which would only serve to slow development on the more important aspects of Wave Function Collapse. Nut-WFC is a fascinating way to add immersion and fluidity to generated levels, but it simply is not required for this project.

2.2 – Townscaper

Townscaper (Stålberg, 2021) is a sandbox city builder, where the player can place or remove blocks that turn into buildings, roofs, or walls based on where other blocks are placed near them. The town gradually gets filled with decorations and parks as the player builds more.

The game uses Wave Function Collapse not to generate a level, but to generate what a given block will look like based on its neighbors, and other context. An example of this is when buildings are surrounding a flat surface, a courtyard is created in the middle.

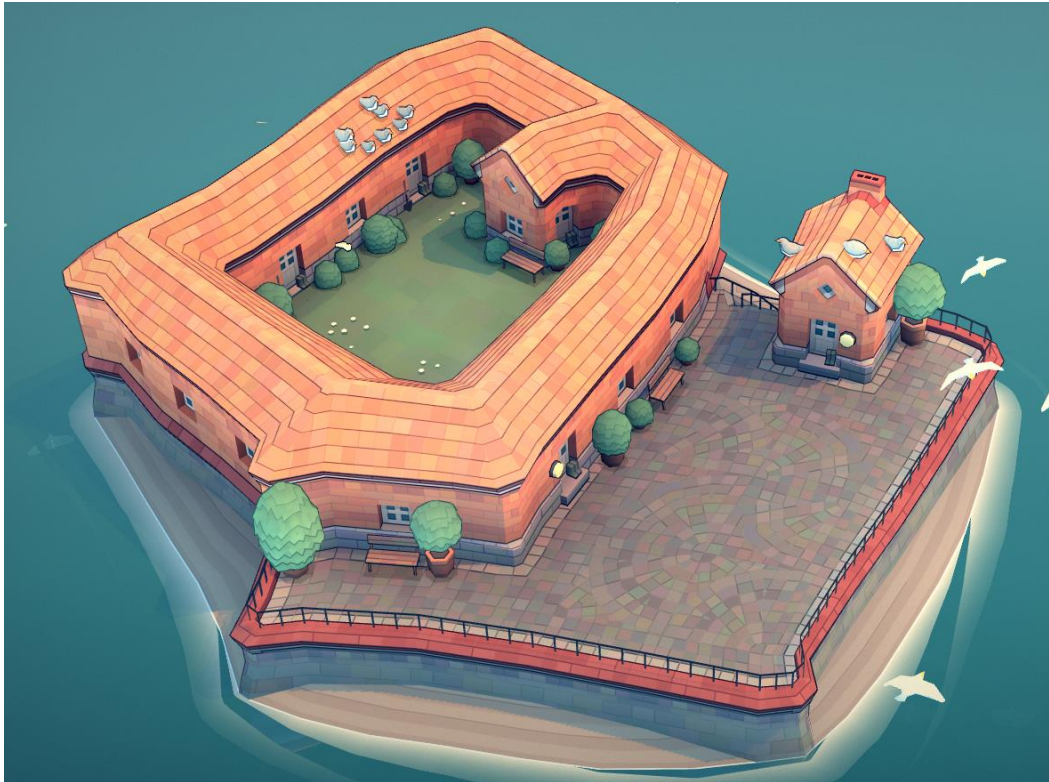


Figure 2-2 A Screenshot of Townscaper's dynamic visual style using Wave Function Collapse

Using Wave Function Collapse to determine the visual style of a slot in the world is a very interesting use of the algorithm and goes against what many are thinking when hearing about it. The method itself is not required for the project artefact, but it may be helpful to keep in mind that the algorithm can be used for more than simply generating levels. Reading about how townscaper implements WFC is also helpful for figuring out how to implement a personalised version of the algorithm for this project.

2.3 – Design-Level Constraints

The paper *Enhancing Wave Function Collapse with Design-level Constraints* by Sandhu, Chen and Mckoy (2019) discusses implementing “Design-Level Constraints” into WFC.

The constraints mentioned in the paper are more technical constraints, that affect how the map generation functions, rather than allowing for potential level designers to tweak values to affect the level generation. An

example of a constraint introduced in this paper is Weight Recalculation, which alters the probability space to forcefully make certain tiles more likely to appear within an area. This happens at runtime with the use of a trigger when a cell gets collapsed. Weight Recalculation is not necessary for this project, but cell weights will be an invaluable tool to adjust the generation of levels.

Another constraint mentioned in the paper is Area Propagation, a modification to the algorithm that allows certain cells to trigger events in an area when they are collapsed. An example of this is if a Wave Function Collapse algorithm generates a tile with a torch, the surrounding area should be altered to only have tiles associated with the light, rather than the dark. This allows for “Sub-Areas” to be made, within which the ruleset is different. This is a fascinating constraint, would not be immensely helpful for testing map diversity, as sub-areas would potentially get in the way of showing trends within rulesets, or would distract participants in the survey away from the more relevant aspects of the levels they need to compare.

Chapter 3 Methodology

This chapter will give an in-depth explanation for how the project artefact, and resulting survey, was designed and developed. Additionally, descriptions and reasoning will be given for any technical decisions, or design choices as to allow for the method to be critically analysed and replicated by future researchers. Finally, any challenges that were encountered will be documented so that they may be avoided in future work.

3.1 - Overview

To sufficiently answer the research question, an application was implemented into Unity that demonstrates Wave Function Collapse. The main purpose of the application was to generate worlds using WFC and control the outputted level by tweaking values in the accompanying design constraints. These generated levels would then be put into a survey, to measure if they are adequately diverse.

The WFC algorithm itself was developed first, with design constraints being implemented afterwards as additional components to attach to the Unity game object that contains the algorithm, or as variables within each individual cell.

In the survey, generated levels would be compared to others which may or may not have the same design constraints active during generation. This allows for the levels to be qualitatively measured, to get a baseline understanding of if the levels are subjectively seen as “diverse” from the other generated levels. These survey results will then be analysed, and a conclusion will be drawn about whether the given implementation is able to maintain diverse levels despite the design constraints applied to the generated levels.

3.2 - Preparation

So that the Wave Function Collapse algorithm can be implemented correctly, it was essential for proper research to be done into the algorithm and its history. There were primarily 2 games which stuck out for its usage of WFC, being *Bad North* and *Townscaper*. These utilised the algorithm in different ways, so it was helpful for expanding the potential ways to take the implementation for this project.

From researching other implementations of WFC, the following decisions were made about the design of the project artefact:

- The algorithm would generate a large, but not excessive map. A two-dimensional, 10x10 grid of square cells would suffice.
- It needs to be easy to add or remove cells to generated maps, as to allow for making different maps with the same algorithm.
- It must be simple to enable or disable design constraints, as well as adjust parameters within said constraints.

With a rough idea of how to implement Wave Function Collapse, an important decision must be made about what game engine will be used for the development of the algorithm.

For the project artefact, Unity was chosen for a couple reasons.

Firstly, prior experience with the engine means that time spent implementing will be dramatically reduced, and less time needs to be spent learning how the engine works.

Additionally, many features of the main alternative, Unreal Engine, are not needed for this artefact. Being able to run the algorithm on a lower end computer such as a laptop is good for productivity and portability. An engine such as Unreal is simply too powerful for what the project needs. Finally, Unreal Engine can be time consuming and difficult to use. This combined with lack of experience with the engine would make implementing the algorithm a more tedious and slow process.

Godot was also considered as a possible option, but due to lack of experience with the engine, it appeared less ideal than Unity. It also has the same reasoning as Unreal Engine, with time spent learning the engine potentially getting in the way of the actual implementation of WFC.

Some research needed to be done on technical details regarding the implementation of Wave Function Collapse. The video titled “EPC2018 | Wave Function Collapse in Bad North | Oskar Stalberg” (YouTube, 2018) has an in-depth explanation for how WFC is employed in *Bad North*, which was an invaluable reference.

3.3 – Application Design

Wave Function Collapse as an algorithm can be implemented in many ways. These usually fall into one of two categories.

One method is showing the algorithm a map, which it then analyses, determining rules that the inputted map follows. It then generates a different map that follows the same rules as the input. This method can work well, but if the input map is too small, or the tile set is inadequately designed, the resulting maps can be nonsensical or unusable. It also cannot understand design constraints, as they can be a lot more complex than simple adjacency rules.

The alternative is to write the rules the algorithm needs to follow, directly into the algorithm itself. This method has the benefit of allowing very specific details to be made clear and allows for design constraints to be implemented directly as well. All of this leads to this method being very consistent, but with the drawback of a longer time to implement.

Because the artefact requires the ability to implement design constraints, the latter option was deemed ideal for this project.

Many implementations of WFC have cells keep track of what other cells they can be placed next to. This, however, can make it difficult to add

new cells, as every cell would need to keep track of every other cell that is compatible with it. Many of these implementations also do not allow rotations or mirroring of cells, which can be useful for memory efficiency, as well as deployment time for new cells. For these reasons, a new method of handling cells has been devised, named the “Cell and Port Method”.

3.3.1 – Design of Cells and Ports

In Wave Function Collapse, a cell would be something like a straight road, with grass on either side. Each cell needs to have a port for each edge, so on a two-dimensional grid, each cell has 4 ports. For a straight road tile, the road would continue in a straight line, so the ends where the road continues would have a port named “Road”. There would also be “Grass” ports on either side of the road. In this example, these two ports would only connect with identical ports, roads with roads, and grass with grass. With this minimalistic ruleset, you can generate an infinitely long stretch of straight roads with grass between them.

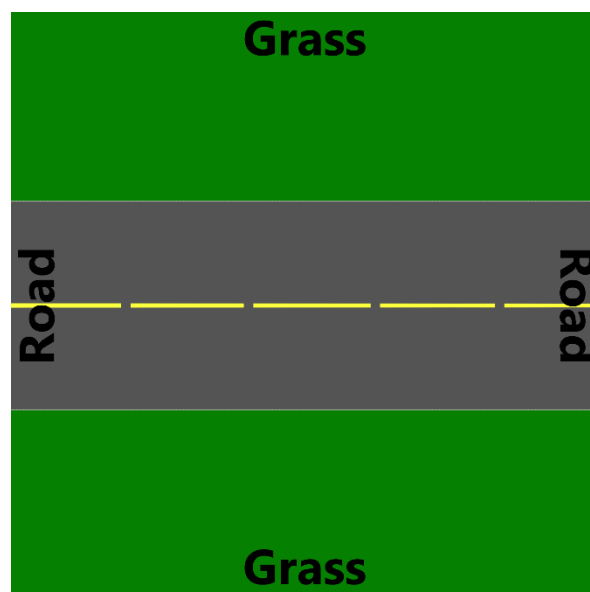


Figure 3-1 An Illustration of a straight road cell, with each edge's port displayed

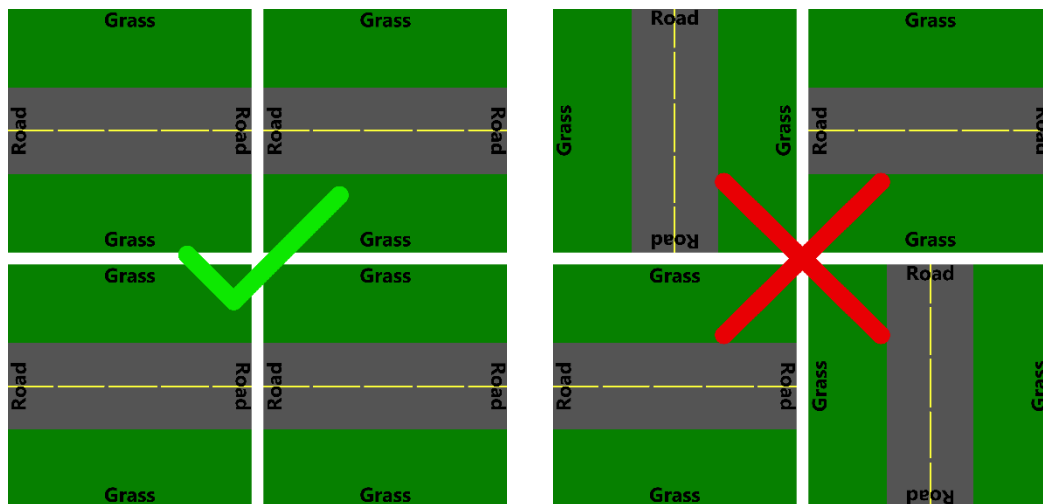


Figure 3-2 Two Illustrations of 4 identical cells being placed next to each other. The first example fulfils the rules, because the ports that are connected are the same. The second example does not, because road and grass ports are not allowed to connect in this example

So that data about cells and ports may be stored and used by the algorithm, a data container must be decided for them. For this purpose, Unity's scriptable objects seemed fitting, as they allow for keeping both variables and functions, but can also be created and stored as asset files, which exist outside of the scene. This proves to be more efficient for memory than prefabs, as accessing the data is done by reference rather than copies.

With this rough idea in mind, a more detailed plan for cells and ports can be developed, starting with cells. Every cell needs to have a visual component to them, as to make a functioning map. This can either be a sprite, or a mesh, out of which the latter was chosen for this project, as it allows for potentially expanding the level generation into the third dimension if required. This decision also allows for viewing the maps in a three-dimensional space, which may prove helpful for survey participants to better determine similarities and differences between generated maps.

Functionally, each cell should be able to rotate if needed, as many potential cells are identical to others when rotated, so making them all be considered 1 cell can massively improve time efficiency for implementation. With the same logic, mirrored cells should be an option if a cell isn't symmetrical, allowing each cell to have a potential of 8

different cells that they add to the pool which the algorithm draws from when generating. To allow for fine tuning of individual cell frequencies, cell weights are also needed. Finally, each cell needs a reference to a port, for each edge, for a total of 4 ports.

Ports are not as detailed as cells, only requiring a couple variables to function. Each port needs to keep track of which ports they can connect to, so a list of references must be implemented. They are unchanged when a cell rotates and therefore does not need to account for it. When mirroring a cell, however, all parts of the cell are mirrored, which includes the edges. This means that ports need to account for whether they are mirrored or not. For this purpose, a reference to another port needs to be stored within each port, so that if an edge is not symmetrical, the port knows what different port to treat itself as if the cell it belongs to is mirrored.

To allow for storing rotation and mirror data in each individual cell, a class “CellInfo” is made, which stores a reference to the cell type it represents, as well as an integer for its rotation, where a 0 means unchanged rotation, and a 3 means rotated clockwise 90°, 3 times. A boolean is also stored for its potential mirroring. This class is what primarily gets used by the algorithm when generating maps, so it also has a function for returning the port that a given direction will encounter, while keeping in mind its own rotation and mirroring:

```
public Port GetPort(int direction, int rotation = 0, bool mirrored = false) {
    if (mirrored) {
        if (direction == 1 || direction == 3) {
            direction += 2;
            direction = direction % 4;
        }
    }
    int rotatedPortID = (direction - rotation + 4) % 4;
    return mirrored ? ports[rotatedPortID].Mirror() : ports[rotatedPortID];
}
```

Note that direction is also an integer, and functions similarly to rotation. A direction of 0 is equivalent to north, while a direction of 1 is east, 2 is south, and 3 is west.

With this framework in place, cells and ports can be created easily, and time spent implementing is spent as efficiently as possible. Additionally,

seeing cells listed in engine will be easier to understand, due to the lack of repeat cells. Overall, it will benefit the remainder of the project development greatly.

3.3.2 – Design of the Map

The map is what contains the generated level, but it is also used to keep track of where each cell is being placed relative to each other. Some variables were added to adjust the number of slots it has on the x and y axis, but the dimensions of any levels generated for the project was 10x10. Additionally, a two-dimensional vector is stored to use for scaling the generated level by, as it needs to be placed in the scene to be properly viewed.

Due to the algorithm being so iterative, and repeating steps often to accomplish its goal, it is critical to save as much processing time as possible, by optimizing the code. This is something that is kept in mind while designing the map, as well as the algorithm itself, which is described later.

Because it is a Wave Function Collapse algorithm, each slot on the map must have the potential to become any cell that is input into the pool. To accommodate this, a new class must be created, named “QuantumCell”, to represent one slot before it is fully decided, or collapsed. The quantum cell has a list of Cell info objects, one for each possible state. This list is intended to be reduced as states become impossible due to local constraints. It also has a boolean so that the algorithm can check if quantum cells have been collapsed immediately, rather than having to check the list every time.

The map itself has a two-dimensional array of quantum cells. Having them in an array makes it simple and intuitive to see which quantum cells are adjacent to which, by changing the value of a coordinate by 1.

To determine what cell to collapse next, following a “Lowest entropy first” heuristic is intuitive, because humans tend to prefer choosing between a smaller number of options. This is due to *choice overload*, as outlined in the paper “When Choice is Demotivating: Can One Desire Too Much of a Good Thing?” by Iyengar & Lepper (2000). Potentially due to this phenomenon, it is speculated that less choices would result in less contradictions during level generation by Heaton (2018) on his blog, although this has not been proven. Regardless, this heuristic was chosen for the algorithm to use when deciding which cell to collapse next. In the case of multiple quantum cells having an equal number of states, it is picked at random:

```
public Vector2Int LeastEntropy() {
    List<Vector2Int> min = new List<Vector2Int>();
    int size = int.MaxValue;

    for (int i = 0; i < slots.GetLength(0); i++) { //For every element in 2D array
        for (int j = 0; j < slots.GetLength(1); j++) {
            if (slots[i, j].collapsed) { //If cell is collapsed, ignore
                continue;
            }
            if (slots[i, j].states.Count < size) {
                //If smaller than the current size, overwrite the previous list and
                //set size to be equal to its length
                min.Clear();
                size = slots[i, j].states.Count;
                min.Add(new Vector2Int(i, j));
            } else if (slots[i, j].states.Count == size) {
                //If equal to current size, add itself to the list
                min.Add(new Vector2Int(i, j));
            }
        }
    }

    if (size == int.MaxValue) {
        //If size hasn't changed, all cells must have collapsed
        return Vector2Int.left;
    }

    Vector2Int chosenCoords = min[Random.Range(0, min.Count)];
    //Pick a random element of those that are smallest

    return chosenCoords;
}
```

The map also contains a function for collapsing a cell at a specific coordinate. Due to cell weights needing to be considered, this is done by adding all available cells' weights together, then picking a random float from the total weight. Then, the algorithm can count the weights again, iterating up the index until the randomized float value is reached:

```

public CellInfo CollapseCell(Vector2Int coords) {
    List<CellInfo> allStates = new List<CellInfo>(slots[coords.x,
coords.y].states);
    int chosenStateIndex = 0;

    //Count together all weights in order so that it can be recounted later
    float totalWeight = 0;
    for (int i = 0; i < allStates.Count; i++) {
        totalWeight += allStates[i].cell.weight;
    }

    //Pick a random number within the range of all weights
    float randomWeightIndex = Random.Range(0, totalWeight);

    //Then find out where in the list of weights (in order) the random number is,
    giving us the randomly picked cell while keeping in mind weightings
    float countingWeight = 0;
    for (int i = 0; i < allStates.Count; i++) {
        countingWeight += allStates[i].cell.weight;
        if (randomWeightIndex < countingWeight) {
            chosenStateIndex = i;
            break;
        }
    }

    if (countingWeight == 0) {
        Debug.Log("0 Hit");
    }

    slots[coords.x, coords.y].states.Clear();

    slots[coords.x, coords.y].states.Add(allStates[chosenStateIndex]);
    slots[coords.x, coords.y].collapsed = true;

    return slots[coords.x, coords.y].states[0];
}

```

Finally, the level needs to be displayed in an efficient manner. Since the algorithm uses meshes, meshes for each cell were created in Blender for this project.

When the algorithm has collapsed every quantum cell, it takes the mesh from each, and using the CombineMeshes() Unity function, combines them into one final mesh, which is given to the game object containing the algorithm:

```

public Mesh PlaceMap() {
    CombineInstance[] instances = new CombineInstance[x * y];
    slots2 = new CellInfo[x * y];

    for (int i = 0; i < slots.GetLength(0); i++) {
        for (int j = 0; j < slots.GetLength(1); j++) {

            int index = i * x + j;
            Matrix4x4 pos = Matrix4x4.Translate(new Vector3(size.x * i, 0, -size.y
* j));
            Matrix4x4 scaler = Matrix4x4.Scale(meshScale);

            slots2[index] = slots[i, j].states[0];

            instances[index] = new CombineInstance {
                mesh = slots[i, j].states[0].GetMesh(),
                transform = pos * scaler
            };
        }

    }

    Mesh finalMesh = new Mesh();
    finalMesh.CombineMeshes(instances);

    return finalMesh;
}

```

3.3.3 – Design of the Algorithm

The Wave Function Collapse algorithm is the container for the map, as well as being where the list of cells to be used in the algorithm is stored. It also has a list of cell info objects named “allTrueCells”, which takes the pool of cells to be used, and expands it into every variation of every cell, so that they can be used in the algorithm. With these, it can start to generate levels and place them into the map.

The Wave Function Collapse algorithm functions inside a loop, that is repeated until the map is fully generated. First, it calls the “LeastEntropy()” function on the map and collapses the chosen quantum cell. Once that cell is collapsed, adjacent quantum cells will likely have states that would result in a contradiction, so they must be accounted for. The way the algorithm handles this is by putting all adjacent quantum cells into a queue, and for every element in the queue, they get checked to see what states are no longer compatible. If any state is removed from these quantum cells, all adjacent cells will also be put in the queue. This process is repeated, until states are no longer removed. Finally, the algorithm repeats, picking a new cell with the lowest entropy. Once all

quantum cells are collapsed, the map has been fully generated and can be placed in the scene.

```
//The main WFC Loop
while (true) {
    queue = new List<Vector2Int>(); //Queue for what tiles to check
    queueHash = new HashSet<Vector2Int>(); //Queue for what tiles to check

    Vector2Int minEntropyCoords = map.LeastEntropy();
    if (minEntropyCoords == Vector2Int.left) {
        //Vector2Int.left means that all cells are collapsed, and map generation
        //cannot continue
        break;
    }
    CellInfo currentCellInfo = map.CollapseCell(minEntropyCoords);
    queue.Add(minEntropyCoords);
    queueHash.Add(minEntropyCoords);

    while (queue.Count > 0) {
        CheckEdges(queue[0]);
        queueHash.Remove(queue[0]);
        queue.RemoveAt(0);
    }
}
```

The queue that keeps track of what cells need to be checked, has two components to it, a list of coordinates, and a hash set of coordinates. These 2 containers hold the same data, but for different reasons. The list allows for using the index to check cells in the order they were added, removing the element at index 0, which moves all elements down by 1. By checking cells in that order, the cells most affected by a cell's collapse will have those effects take place immediately, with less affected cells taking place later as the effects propagate outwards. When cells start to no longer get affected, the vast majority of affected states have already been removed.

The hash set has a benefit over lists, being that it is quick to check if a hash set contains an element, being in $O(1)$ time complexity. Since quantum cells are likely to be affected by more than one of their neighbours during propagation, this can be used to check if they already are in the queue. If a quantum cell is checked twice, nothing will change on the second check, making it a waste of time, which is why the algorithm has the hash set, to avoid those situations.

When a quantum cell is placed in the queue and checked, it is done by checking all 4 edges of that cell. If a cell is located at the edges of the map, some edges are empty space and can be ignored. The

“CheckEdges()” function takes the coordinates of the checked cell as a parameter, and calls the “CheckEdge()” function 4 times. This function has 2 parameters, one for the coordinates of the chosen cell, and one for the coordinate offset, being 0 in one axis, and a 1 or -1 in the other.

```
public void CheckEdges(Vector2Int coords) {
    if (coords.y != 0) { //Check top if not at the top edge of the map
        CheckEdge(coords, new Vector2Int(0, -1));
    }

    if (coords.x != map.x - 1) { //Check right if not at the right edge of the map
        CheckEdge(coords, new Vector2Int(1, 0));
    }

    if (coords.y != map.y - 1) { //Check bottom if not at the bottom edge of the map
        CheckEdge(coords, new Vector2Int(0, 1));
    }

    if (coords.x != 0) { //Check left if not at the left edge of the map
        CheckEdge(coords, new Vector2Int(-1, 0));
    }

    //I need to remove any state that can no longer occur in *all* possible
    //scenarios, not *any* scenario.
    //Obviously when a cell is collapsed, that is the only possibility
}
```

Inside the “CheckEdge()” function, the coordinate offset is used to find the coordinates of the adjacent cell, but it also serves to find the direction integer that is needed for other functions. This could be done with a couple if of statements but considering that this function is run 4 times for every cell that is put in the queue, which happens potentially thousands of times throughout the generation of a level, a slight optimization was made. It is possible to calculate mathematically, an integer between 0 and 3 for the direction, by using the coordinate offset given as a parameter. A direction of 0 means north, 1 means east, 2 means south, and 3 means west:

```
//Translates an offset in Vector2 format into a direction integer where:
//north is 0, east is 1, south is 2, and west is 3.
int directionComponentX = (targetOffset.x - targetOffset.x * 2) +
(Math.Abs(targetOffset.x) * 2);
//An offset.x of 0 returns 0, -1 returns 3, 1 returns 1

int directionComponentY = targetOffset.y + Math.Abs(targetOffset.y);
//An offset.y of -1 and 0 returns 0, and 1 returns 2

int dir = directionComponentX + directionComponentY;

//Calculates the opposite direction for the target cell to find its Port
int targetDirComponentX = (-targetOffset.x + targetOffset.x * 2) +
(Math.Abs(targetOffset.x) * 2);
//An offset.x of 0 returns 0, -1 returns 1, 1 returns 3

int targetDirComponentY = -targetOffset.y + Math.Abs(targetOffset.y);
//An offset.y of 1 and 0 returns 0, and -1 returns 2

int targetDir = targetDirComponentX + targetDirComponentY;
```

Once a direction integer has been calculated, the edge can be analysed. A second hash set is created to keep track of which states are deemed impossible, as this is only fully decided once all the states of both the currently checked cell, and the adjacent target cell, have been compared fully. The function iterates through both lists, comparing the touching ports for every combination. Because a state in the target cell must be incompatible with every state in the current cell for it to be deemed impossible, target states are only added to the hash set when they are being compared to the first state in the current, since if they are compatible with this first state, that target state cannot be impossible. While iterating through all later combinations of states, any time two ports are compatible, that state gets removed from the hash set if it is present, as that state is no longer impossible. Once all combinations of states have been checked, those deemed impossible are removed from the target's quantum cell states:

```
if (impossibleCells.Count > 0) {
    bool hasRemovedState = false;
    for (int i = map.slots[targetCoords.x, targetCoords.y].states.Count; i > 0; i--) {
        if (impossibleCells.Contains(map.slots[targetCoords.x, targetCoords.y].states[i - 1])) {
            //Remove all states in the target cell that is in the hashSet
            map.slots[targetCoords.x, targetCoords.y].states.RemoveAt(i - 1);
            hasRemovedState = true;
        }
    }
    if (hasRemovedState && !queueHash.Contains(targetCoords)) {
        //If any state of the target cell was removed, it must be added to queue to check if it propagates
        queue.Add(targetCoords);
        queueHash.Add(targetCoords);
    }
}
```

When two states are being checked, the current cell retrieves the reference to its port through the “GetPort()” function, which takes a direction integer as a parameter. This port runs its “ComparePort()” method, which checks if the port is compatible with the port given as a parameter. The parameter in this case is the target state's port, which takes in the opposite direction integer:

```
if (!self.GetPort(dir).ComparePort(target.GetPort(targetDir)))
```

Overall, the Wave Function Collapse algorithm satisfies the conditions to generate a vast number of different levels and allowing new cell types to be introduced with ease. The level generation speed has not been measured, as it does not aid in answering the research question, however, it generates levels near instantly when observed by the human eye, which will more than suffice for testing and generating levels for the survey.

3.3.4 – Design of the Coordinate Constraint

With the base algorithm implemented, work could begin on augmenting it, by implementing design constraints. Due to time constraints, only 1 design constraint could be implemented by the time the survey needed to be held, that being coordinate constraints. Coordinate constraints are restrictions on what coordinates a specific cell type can be found in, such as only permitting tunnel cells to appear below a Y level of 5.

To connect the coordinate constraint to the base implementation of Wave Function Collapse, it was decided to use Unity's Entity Component System to make the constraint a component that could be added to the game object with ease. This also allows for having variables separate from the base WFC component. The coordinate constraint activates only at the beginning of level generation, before any of the base algorithm has begun. This will be nicknamed "The Pruning Stage", due to the fact that any forbidden cell states are pruned from their quantum cell.

The only variable that a level designer would interact with when using the component, is a list of "CellConstraint"s, a class made for this purpose. They contain 2 lists within them, one of which is a list of Cells, representing what cells are being affected by the constraint. The other list is another new class, named simply "Constraint". Each constraint contains an enumerator for the axis the constraint affects, being X or Y, a different enumerator for whether the constraint allows cells above, below, equal or not equal to the third variable, an integer named "Value":

```

[SerializeField] List<CellConstraint> constraints = new List<CellConstraint>();

public enum ComparisonType {
    Over,
    Under,
    Equal,
    Not
}

public enum Coordinate {
    X,
    Y
}

[Serializable]
public class Constraint {
    [SerializeField] public Coordinate coordinate = Coordinate.X;
    [SerializeField] public ComparisonType comparison = ComparisonType.Over;
    [SerializeField] public int value = 0;
}

[Serializable]
public class CellConstraint {
    [SerializeField] public List<Cell> ConstrainedCell = new List<Cell>();
    [SerializeField] public List<Constraint> Constraints = new List<Constraint>();
}

```

The reason for the component having lists within lists, is to allow for very specific rules to be made for cells. Each constraint in the same scope, or list, are all considered together, and all of them must align to allow an affected cell to exist in any given slot on the map. In other words, each constraint in the same list is added together, as if using an AND statement in boolean logic.

For any number of cell constraints within the component, only one of them needs to be fulfilled to allow an affected cell to exist, in other words, they are functionally OR statements within boolean logic:

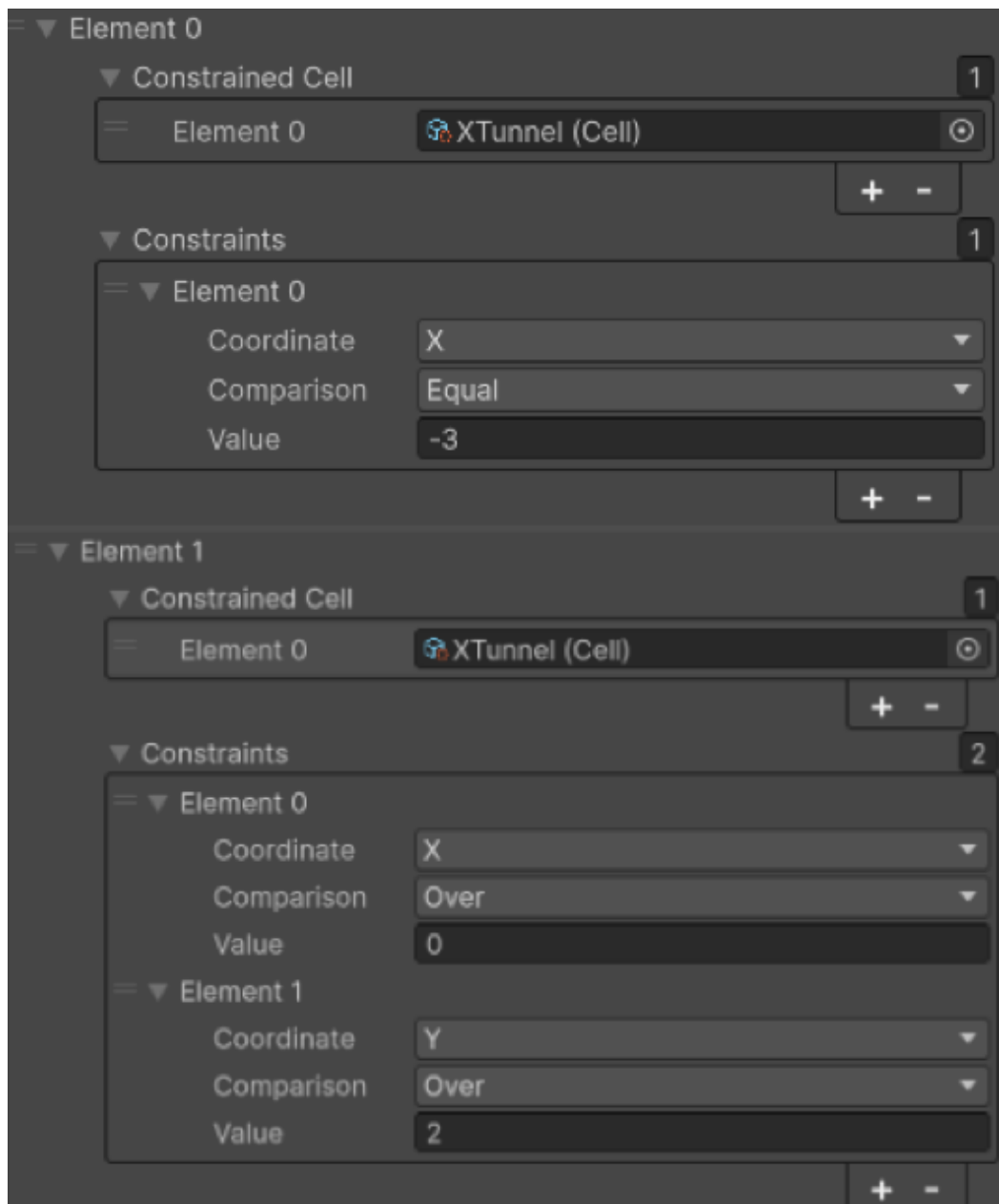


Figure 3-3 A screenshot of a unity component, In this example, the XTunnel cell can either show up anywhere where X is equal to -3, OR it can show up anywhere where X is more than 0, AND Y is over 2.

Inside the script, the coordinate constraint has 4 hash sets to allow it to function. It has a local allow list, as well as a local deny list. Both get used to keep track of which quantum cell states are permitted by the rules. When all the constraints within a cell constraint have been processed, any states left in the hash sets are transferred over to the other 2 hash sets, before the local lists get cleared. These new hash sets are named the global allow list, and global deny list. The global deny list is used to remove all the affected cells right before the pruning stage ends, due to the coordinate constraint being a subtractive process.

The global allow list's purpose is to process logic with OR statements, because if a cell is allowed in a different cell constraint, the global allow list would have it placed within it. This means that when checking to add a cell to the global deny list, the global allow list must be checked first, so that any permitted cell states do not removed:

```
private void GlobalListsAdd() {
    foreach (ListStruct cell in localAllowList) {
        if (globalDenyList.Contains(cell)) {
            globalDenyList.Remove(cell);
        }
        if (!globalAllowList.Contains(cell)) {
            globalAllowList.Add(cell);
        }
    }

    foreach (ListStruct cell in localDenyList) {
        if (!globalAllowList.Contains(cell) && !globalDenyList.Contains(cell)) {
            globalDenyList.Add(cell);
        }
    }
} //The ListStruct class is a CellInfo, combined with an X and Y coordinate
```

3.3.5 – Design of the Survey

For the survey to fulfil its purpose, it needs to display levels to participants, that are in some subjective metric, diverse. If multiple participants can easily classify levels, they must share some consistent, identifiable features. One can infer that these features imply that the levels within that ruleset are less diverse. A survey that contains multiple different rulesets that generated levels use must be made, so that various rulesets can be compared together.

The tile set that the algorithm will use for the survey, is meant to represent a cave with ponds and rivers inside. It has flat terrain, flat terrain with rocks, low terrain that is covered with water, and finally edges that connect the higher terrain to the lower, water terrain. The cell weights are as follows:

- Flat terrain: 4
- Rocks: 0.5
- Low terrain: 0.6
- All edges: 0.6



Figure 3-4 A map generated without constraints. Note that two opposing edges can connect to each other, making a thin stream of water

The first ruleset that was made, uses the coordinate constraint to limit any low terrain tile to only appear on the outer edges of the level. This decreases the number of low terrain tiles in the map dramatically, so an adjustment was made in the weighting of low terrain. Low terrain tiles had their weight increases from 0.6, all the way to 4. It has been appropriately named "WaterEdge". Due to regular edges being unaffected, thin streams still appear all over the generated map:



Figure 3-5 A map generated with the "WaterEdge" ruleset

Another ruleset that affects the rock tiles was created, named “LeftRocks”. This ruleset prohibits rock tiles from appearing anywhere where the X coordinate is below 6. This makes those sections of the map very flat and empty:

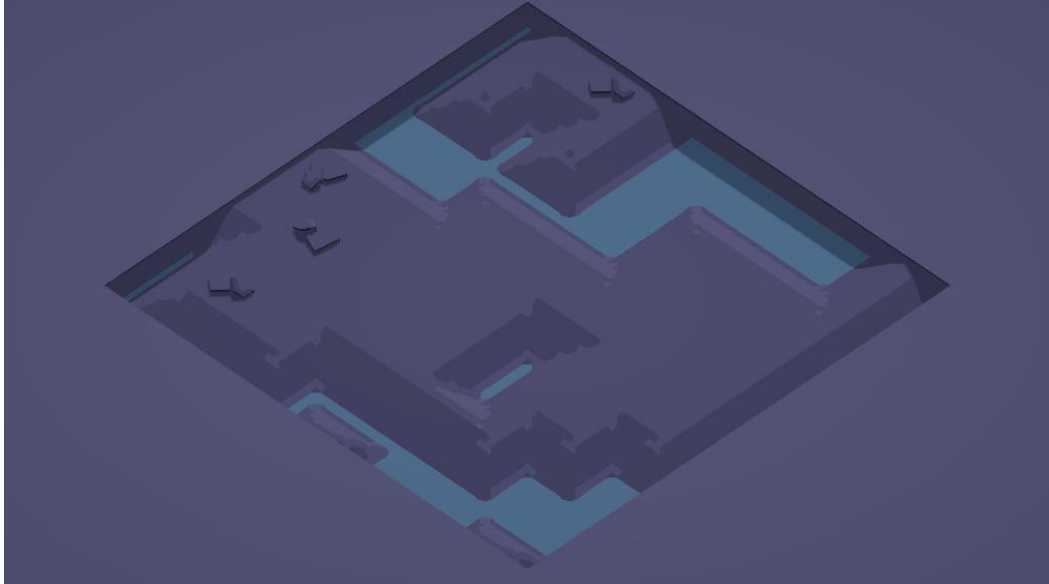


Figure 3-6 A map generated with the "LeftRocks" ruleset

Due to there not being too many options for how to change level generation in a meaningful way with just the coordinate constraint, the next ruleset exclusively uses cell weights. Named “NoOuterEdge”, the ruleset makes the outer corner edge tiles have a weight of 0.2 instead of 0.6 and makes the inner corner edge tiles have a weight of 2 instead of 0.6. This change prevents most bodies of water to generate with enough

volume to be seen as more than a river or puddle, and reveals a trend of small “1x1” puddles:



Figure 3-7 A map generated with the "NoOuterEdge" ruleset

In a similar vein, the “NoInnerEdge” ruleset does the inverse. Changing the inner and outer edges’ weights from 0.6, to instead be 0.2 and 2 respectively. This ruleset has a similar trend, where small “1x1” islands appear in the generated levels:

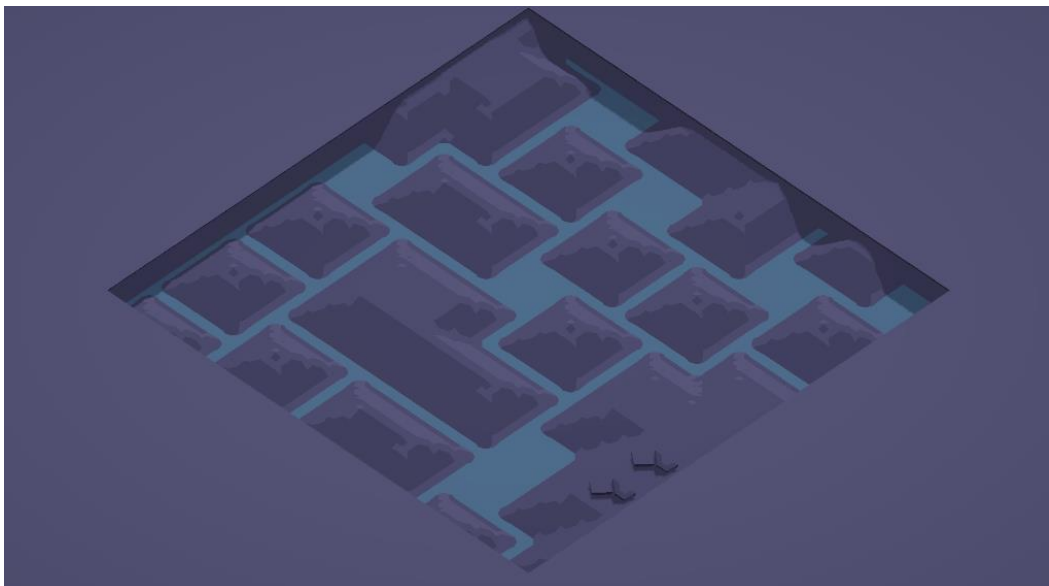


Figure 3-8 A map generated with the "NoInnerEdge" ruleset

With the rulesets to use for the survey decided, work can be done on designing the questions themselves, which is done within google forms.

To allow for comparing between different rulesets, each question will be repeated several times within a section, but with different rulesets being used. This is done so that each time a ruleset is used is different than previous questions, 10 levels have been generated with every ruleset, except for “WaterEdge”, which has 6. This irregularity is due to the number of questions and number of levels not aligning, so some levels had to be cut. There is an argument to be made for cutting some of each ruleset, rather than all from one, but this is refuted by the fact that each ruleset needs multiple levels to be able to make a set, which is explained in the following paragraph.

The first section provides the participants with a single level that has been generated with one of the rulesets, as well as a set of 4 levels that have all been generated using the same ruleset. The set of levels may or may not be using the same ruleset as the single level. The participant can answer that it does, does not, or that they are unsure about whether the single level “fits” with the level set, which is another way of saying that they share the same ruleset. Below each question, the participant is asked why they answered what they did, before the question repeats with new levels. The question is asked for 7 different single levels, and 3 different sets of levels.

The reasoning behind the first section is as follows:

If most participants can correctly identify what levels match the set, and which do not, that means that the constraints are visible, which can be either good or bad. If the accuracy is too high, however, it can imply that the level generation is generating levels that are too similar, making the levels less diverse within each ruleset.

If participants are primarily incorrect with their decision, one can infer that there is little cohesion between the generated levels. This outcome can make it difficult to draw a conclusion, so is not ideal.

If there is a balance between correct and incorrect guesses, and a general trend of being unsure, it implies that the design constraints are

guiding the participant into seeing a theme or trend, while still leaving room for diversity between the generated levels.

The second section provides the participants with 1 single level again, but this time provides them with 2 different sets of 3 levels each. The participant is asked to determine whether the single level matches set 1 or set 2. They can also answer that they are unsure if needed. Each question is followed by another text box asking for why the participant answered what they did, before repeating. This question is asked 9 times, with 9 different single levels, and 3 different pairs of sets of levels.

The second section is checking a slightly different metric than the first: If participants mostly have a high accuracy, each ruleset displays a recognisable trend, which may suggest that the design constraints narrow down level diversity, but could instead imply that each ruleset is clearly distinguishable from each other.

If the participants have a low accuracy, it means that all the levels look similar to each other, despite having different rulesets. This suggests that the constraints do not affect level diversity.

Overall, the survey is designed in a way where, to prove the hypothesis, responses must be correct often enough to show that the design constraints are affecting the levels at all, but avoiding being overwhelmingly accurate, as that implies that the ruleset makes all generated levels seem too similar, or in other words, have low diversity.

Chapter 4 Results

This chapter will present all results from the project survey. Because the survey is the only source of data for this project, all the data is qualitative. Each participant was provided with a link to the google form. The survey questions can be found in Appendix A.

4.1 – Survey Results from Section 1

Keep in mind that answering the Why / Why not section is optional.

1. Does the level on left the follow the same rules as the set of levels on the right?



Figure 4-1 Question 1 of Section 1 of the Survey

12 Responses

ID ↑	Name	Responses
1	anonymous	The single level seems more connected and prioritises landmass over water. The set of levels use less connected 1x1 tiles.
2	anonymous	Differing heights and land spacing
3	anonymous	The set seems to contain small islands, while the single level is one big island
4	anonymous	Single level is made up of larger groups, the set is made or smaller rectangles
5	anonymous	The level on the left seems to consist of a single connected piece of land, whereas levels on the right have many small disconnected squares of land.
6	anonymous	More homogenous
7	anonymous	No spaces between blocks/buildings/islands
8	anonymous	Level set features fully connected negative space. Single level while having negative space touching every border does this via multiple pockets of negative space rather than a big connected section.
9	anonymous	The level on the left features a single 'continent' compared to the smaller blocks/islands of the rights' sets. Idk I'm interpreting the lighter blue as water for context.
10	anonymous	I dont know what are the rules ilike they look similar but the ones on the right have more seperated blocks
11	anonymous	single level is made up of one big level where as the set of levels is made up of multipul smaller levels
12	anonymous	in the single level isolated islands of land aren't generated

Figure 4-2 Why / Why Not? for Question 1 of Section 1 of the Survey

The consensus is that the single level appears more connected, having one single island, while the set of levels all have many individual islands, with less connected land. The majority of responses are correct.

3. Does the level on left the follow the same rules as the set of levels on the right?



Figure 4-3 Question 2 of Section 1 of the Survey

10 Responses

ID ↑	Name	Responses
1	anonymous	All levels have a variety of 1x1 island tiles mixed with one or two larger connected islands.
2	anonymous	Chunk of land, line of land and individual pieces
3	anonymous	Same reasoning as before
4	anonymous	It feels more consistent, has larger regular rectangles, with a larger object
5	anonymous	The single level looks similar to the top left level in the set of levels. However, it seems to have less space in-between land platforms compared to the other three examples in the level set.
6	anonymous	Fewer discrete islands
7	anonymous	all of them feature multiple sizes of "island" with large connected negative space that touches all 4 border walls
8	anonymous	Similar generation with some islands and maybe a longer winding portion. Although the landmass on the left generation is quite long compared to the one on the top left with the set of levels.
9	anonymous	similar amount of empty space (the blue) and similar amount of little blocks
10	anonymous	single level abd set of levels are both made up of smaller islands

Figure 4-4 Why / Why Not? for Question 2 of Section 1 of the Survey

Participants note that all the levels have many 1x1 islands, with some slightly larger landmasses present as well. The majority are correct. The single level has a long-connected strip however, which sways some participants.

5. Does the level on left the follow the same rules as the set of levels on the right?

● Belongs with the set 12
 ● Unsure 0
 ● Does not belong to the set 0

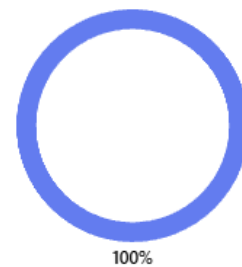


Figure 4-5 Question 3 of Section 1 of the Survey

11 Responses

ID ↑	Name	Responses
1	anonymous	All of the levels are large, connected landmasses with puddles of water throughout.
2	anonymous	Lot of land with holes
3	anonymous	Similar amount of water
4	anonymous	Largely a single mass with small holes
5	anonymous	All of the levels seem to have a single large piece of land containing several puddle-like holes.
6	anonymous	Similar density
7	anonymous	Everything is connected
8	anonymous	all levels have a single connected bit of land touching every border with small holes, while these holes vary in size they all seem within the same range.
9	anonymous	Similar ratio of land to water.
10	anonymous	similar blue/purple ratio and the purple is all grouped together
11	anonymous	all levels are made up of one big island

Figure 4-6 Why / Why Not? for Question 3 of Section 1 of the Survey

All the levels contain a large, connected landmass, with a similar number of small puddles throughout. They also mention a similar ratio between water and land between all the provided levels. Every single participant got this one correctly.

7. Does the level on left the follow the same rules as the set of levels on the right?



Figure 4-7 Question 4 of Section 1 of the Survey

9 Responses

ID ↑	Name	Responses
1	anonymous	While all the levels are large landmasses with puddles, the single level does not include more than a couple tiny puddles, and has three larger ones. The set of levels seem more randomly pierced with smaller puddles.
2	anonymous	Hole shape
3	anonymous	Similar amount of water
4	anonymous	All the levels seem to be made up of one large connected piece of land, containing a few large holes and several small holes.
5	anonymous	Maybe has less discrete pools, too small a slice to tell
6	anonymous	similiar shaped holes within similiar size range
7	anonymous	Looks pretty similar :p
8	anonymous	similar blue/purple ratio and the purple is all grouped together also has similar sized holes
9	anonymous	all levels are made up of one big island

Figure 4-8 Why / Why Not? for Question 4 of Section 1 of the Survey

The single level has fewer small puddles in favour of a couple larger ones, which leaves some respondents unsure, but all levels are made up of a large, connected landmass, and are part of the same set. One notes that the hole shape sticks out.

9. Does the level on left the follow the same rules as the set of levels on the right?



Figure 4-9 Question 5 of Section 1 of the Survey

11 Responses

ID ↑	Name	Responses
1	anonymous	The single level has distinctly more water and a more natural waterflow than the set of levels, which seem more patterned.
2	anonymous	All others have 1x1 holes
3	anonymous	Similar amount of water
4	anonymous	Holes too big
5	anonymous	The holes in the left level seem to be larger and more river-like (longer, irregular shapes), whereas the holes in the levels on the right seem to be smaller and more puddle-like (smaller rectangular shapes).
6	anonymous	Left has higher water/land ratio
7	anonymous	too holey
8	anonymous	Single level has a bit more water compared to the set of levels. While it fits with them, there's something there that is disharmonious when looking at the as a single set.
9	anonymous	Looks similar but there is a square purple island in the middle of the blue bit on the one on the left which there isn't in any of the others
10	anonymous	set of levels are all one big islands whilst the single level is made up of three
11	anonymous	isolated pockets of water aren't generated in the single level

Figure 4-10 Why / Why Not? for Question 5 of Section 1 of the Survey

The single level is observed to not contain any 1x1 holes, and has longer, river-like shapes. The single level also has a higher land to water ratio, one notes. Another says that they all have similar amounts of water. A third mentions that it looks “disharmonious” to view them all as part of the same set. Most of the responses are correctly identifying that they are not part of the same set.

11. Does the level on left the follow the same rules as the set of levels on the right?



Figure 4-11 Question 6 of Section 1 of the Survey

11 Responses

ID ↑	Name	Responses
1	anonymous	The single level has numerous isolated islands and a connected waterway. The set of levels instead has numerous waterways within more connection in the landmass.
2	anonymous	Windy water
3	anonymous	Too much water
4	anonymous	Hole is too well connected
5	anonymous	The levels look pretty similar in terms of land to water distribution, but the behavior of the water doesn't seem the same. In the level on the left, the stream seems to go from one end of the level to the other, and seems to go around small land islands in the middle of the level. The levels on the right differ, the stream seems to be broken off by the land and does not continue from one side of the level to the other. There aren't any island-like platforms in the levels on the right.
6	anonymous	Similarly liney
7	anonymous	level set seems to be connected via 2 size ranges of "rivers" with 1 large landmass. while the single level has the same river style it's made of muliple broken landmasses
8	anonymous	Too many rivers on the left im sus to be real.
9	anonymous	genuinely unsure they all look quite different
10	anonymous	set of level are one big level where as single level is a lot of smaller ones
11	anonymous	all the water is connected in the single example

Figure 4-12 Why / Why Not? for Question 6 of Section 1 of the Survey

Very split opinion, with many saying that the single level has one large, connected body of water, with one saying it is “too well connected”. The set, however, has many large, connected landmasses. Many are unsure. This is the first question where the majority got it wrong.

13. Does the level on left follow the same rules as the set of levels on the right?



Figure 4-13 Question 7 of Section 1 of the Survey

10 Responses

ID ↑	Name	Responses
1	anonymous	All of the levels share similar patterns in their landmass and waterways, such as thicker rivers at the edges of the level and occasional isolated islands.
2	anonymous	Too many thin pieces of water
3	anonymous	Hole seems too thin
4	anonymous	This seems more likely to be apart of the set than the previous question. The level on the left still seems to have more island-like land platforms that aren't connected to a larger piece of land than the levels in the set on the right.
5	anonymous	More discrete islands
6	anonymous	some big rivers some small, large landmasses but also smaller islands.
7	anonymous	Oughr oouhg urgh look same.. i think..
8	anonymous	again they all look quite different - unsure
9	anonymous	set of level are one big level where as single level is a lot of smaller ones
10	anonymous	all the water is connected in the single level

Figure 4-14 Why / Why Not? for Question 7 of Section 1 of the Survey

Again, the participants are quite split. The single level has “too many thin pieces of water”, and has more island-like landmasses that are not connected. The single level has all its water connected, with many islands dotted around the level. One participant notes that the water is thicker on the edges of the map, which is the ruleset for this question. Only 3 of the respondents are correct for the final question in this section.

4.2 – Survey Results from Section 2

15. Which set of levels does the level on the left belong to?



Figure 4-15 Question 1 of Section 2 of the Survey

10 Responses

ID ↑	Name	Responses
1	anonymous	The single level shares a similar ratio of water: land, as well as the pattern of islands and thicker waterways.
2	anonymous	All in set 1 have a larger pool of water with thinner streams
3	anonymous	Holes in set 2 too small
4	anonymous	The land/water distribution in the left level looks very similar to Set 1, whereas Set 2 has very little water in comparison.
5	anonymous	Same water/land ratio
6	anonymous	set 1 has wider water that matches the single level
7	anonymous	Starting with a hard one ok damn. IDK it kinda looks like both.
8	anonymous	amount of blue space is more similar
9	anonymous	single level is on e big island with a few small ones same as set 1
10	anonymous	set two and the single level contain small islands of water whereas set 1 does not

Figure 4-16 Why / Why Not? for Question 1 of Section 2 of the Survey

Participants note the ratio between water and land, as well as the holes sizes being different. There's a strong majority voting for the correct option.

17. Which set of levels does the level on the left belong to?

● Set 1 12
● Set 2 0
● Unsure 0

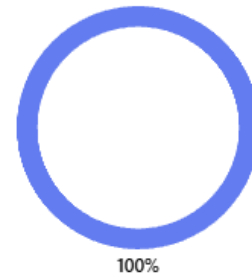


Figure 4-17 Question 2 of Section 2 of the Survey

11 Responses

ID ↑	Name	Responses
1	anonymous	The landmass in the single level is broken up by water in a similar pattern to in set 1.
2	anonymous	More open
3	anonymous	All in set 1 have a larger pool of water with thinner streams
4	anonymous	Holes in set 2 too small
5	anonymous	The land/water distribution in the left level looks more similar to that in Set 1. Additionally, the water streams in the in the left level look long like the ones in Set 1, compared to the smaller puddle-like water holes in Set 2.
6	anonymous	Same water/land ratio
7	anonymous	wide water
8	anonymous	Water ratio and river distribution looks similar.
9	anonymous	direct line from one side to the other which set 2 doesnt have
10	anonymous	single level has more water tahn the set 2
11	anonymous	set two contains small islands of water whereas nether the single level or set 1 do

Figure 4-18 Why / Why Not? for Question 2 of Section 2 of the Survey

Unanimous vote for set 1, the correct set. The landmass is broken up by water, which matches the first set, as well as them all being “more open”. Note that these levels have the “LeftRock” ruleset. It is also pointed out that set 2 has less water, and more tiny puddles of water.

19. Which set of levels does the level on the left belong to?

● Set 1 0
 ● Set 2 12
 ● Unsure 0

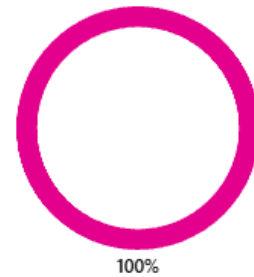


Figure 4-19 Question 3 of Section 2 of the Survey

11 Responses

ID ↑	Name	Responses
1	anonymous	The single level has a similar ratio of water:land to set 2. It also has more isolated puddles like in set 2.
2	anonymous	More land
3	anonymous	Similar amounts of water
4	anonymous	Holes in set 1 too big
5	anonymous	The single level contains many small puddle-like holes, which the levels in Set 2 also contain.
6	anonymous	Same water/land ratio
7	anonymous	thin water
8	anonymous	No rivers :(they took them...
9	anonymous	broken up blue squares similar to set 2
10	anonymous	single level is one big island with minimal water
11	anonymous	set two and the single level contain small islands of water whereas set 1 does not

Figure 4-20 Why / Why Not? for Question 3 of Section 2 of the Survey

Unanimous vote for set 2, which is correct. Participants mention a similar water to land ratio, similarly sized puddles, and a lack of larger bodies of water or rivers.

21. Which set of levels does the level on the left belong to?



Figure 4-21 Question 4 of Section 2 of the Survey

11 Responses

ID ↑	Name	Responses
1	anonymous	Definitely a higher number of isolated islands like in set 2. It is also much more uniform than set 1, which feels more natural.
2	anonymous	More Split
3	anonymous	Contains many small square islands
4	anonymous	Regular boxes, like in set 2
5	anonymous	The single level has many small, square island platforms, very similar to the levels in Set 2. Set 1 is comparatively made up of larger platforms and doesn't seem to follow the same rules.
6	anonymous	Same water/land ratio, also similar number of discrete islands
7	anonymous	lots of water and very square small islands
8	anonymous	'Cause I'm an Island Boy Oh, I'm an Island Boy
9	anonymous	more singular islands
10	anonymous	single level and set 2 both made of little islands
11	anonymous	the single level and set 2 both generate isolated islands of land whereas set 1 does not

Figure 4-22 Why / Why Not? for Question 4 of Section 2 of the Survey

Another unanimous vote for set 2, which is correct, because the single level and set 2 both have many 1x1 islands, and the other set has a more connected landmass.

23. Which set of levels does the level on the left belong to?

● Set 1 0
 ● Set 2 12
 ● Unsure 0

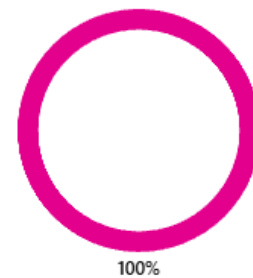


Figure 4-23 Question 5 of Section 2 of the Survey

11 Responses

ID ↑	Name	Responses
1	anonymous	Same reasons as the previous question.
2	anonymous	More split
3	anonymous	Contains many small square islands
4	anonymous	Regular boxes like in set 2
5	anonymous	The single level is has many small square land platforms, which the Set 2 levels also contain.
6	anonymous	Similar number of discrete islands
7	anonymous	characterised by lots of water and small square islands
8	anonymous	No more continent. This generation yearns for islands.
9	anonymous	again more square singular islands in set 2
10	anonymous	single level and set have more water
11	anonymous	the single level and set 2 both generate isolated islands of land whereas set 1 does not

Figure 4-24 Why / Why Not? for Question 5 of Section 2 of the Survey

Every participant voted for set 2, the correct answer, for similar reasons as the previous question. Set 1 does not have as many isolated islands as the single level and set 2.

25. Which set of levels does the level on the left belong to?

● Set 1 12
● Set 2 0
● Unsure 0

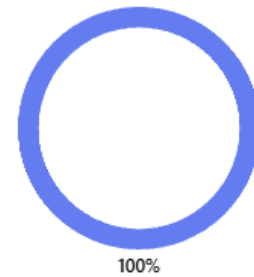


Figure 4-25 Question 6 of Section 2 of the Survey

11 Responses

ID ↑	Name	Responses
1	anonymous	Less isolated landmasses like set 1. The water flow feels more natural too.
2	anonymous	Not split
3	anonymous	Does not contain that many islands
4	anonymous	Set 2 made of regular boxes, the level is more grouped
5	anonymous	The water to land distribution seen in the single level looks more similar to Set 1. Additionally, the single level has several larger land platforms, which can also be seen in the Set 1 levels, whereas set 2 has many small square platforms.
6	anonymous	Similar number of discrete islands
7	anonymous	complex landmass shapes and larger islands
8	anonymous	not enough single squares for set 2.
9	anonymous	set 1 has singular islands but less of like the image
10	anonymous	single level and set 1 made up of larger island and bigger pools
11	anonymous	set 2 both generates isolated islands of land whereas set 1 and the single level do not

Figure 4-26 Why / Why Not? for Question 6 of Section 2 of the Survey

A unanimous vote for set 1, which is also correct. The water “feels more natural”, with less but larger landmasses. One mentions that there is “not enough single squares for set 2”.

27. Which set of levels does the level on the left belong to?

- Set 1 2
- Set 2 1
- Unsure 9

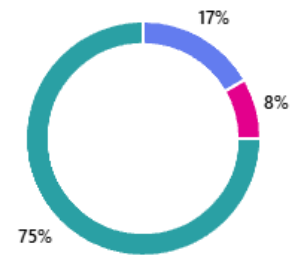


Figure 4-27 Question 7 of Section 2 of the Survey

7 Responses

ID ↑	Name	Responses
1	anonymous	Set 1 and Set 2 look like they could all be from one big set. I'm leaning towards Set 1 because of the similar composition to the first level in Set 1, but every level looks similar here.
2	anonymous	Similar amounts of water
3	anonymous	Set 1 too central
4	anonymous	It honestly looks like it could belong to either set, I can't really see an obvious difference between the sets.
5	anonymous	I got no clue bro
6	anonymous	OK idk it can be both they are very similar.
7	anonymous	these all look the same are you tricking me?

Figure 4-28 Why / Why Not? for Question 7 of Section 2 of the Survey

The majority are unsure, and one suspects that the question is a trick. There is a slight lean towards set 1, the correct option, because of a similar composition in the first level of that set.

29. Which set of levels does the level on the left belong to?



Figure 4-29 Question 8 of Section 2 of the Survey

7 Responses

ID ↑	Name	Responses
1	anonymous	Again, these sets look really similar to me. I'm leaning Set 1 again because of the water and landmass patterns, but the more I look the more I second guess it.
2	anonymous	Set 1 too grouped
3	anonymous	I think it looks maybe a little more similar to Set 1? but I can't really tell the difference between these sets either.
4	anonymous	Similar
5	anonymous	This one is hard too but I think its set 2 based on rivers.
6	anonymous	set 2 has ones with broken square islands and the single level has a couple at the side in the top corner
7	anonymous	looks like set one to me

Figure 4-30 Why / Why Not? for Question 8 of Section 2 of the Survey

Exactly half of the participants answer set 2, with most of the remaining participants being unsure. One notes that set 1 is “too grouped”. Set 2 is the correct option for this question.

31. Which set of levels does the level on the left belong to?

- Set 1 6
- Set 2 1
- Unsure 4

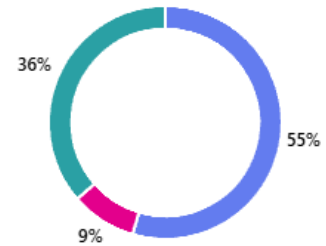


Figure 4-31 Question 9 of Section 2 of the Survey

6 Responses

ID ↑	Name	Responses
1	anonymous	Again, the sets look too similar. I'm leaning towards set 1 once again because of the water pattern.
2	anonymous	Set 2 is too broken up
3	anonymous	I think the land/water distribution looks more similar to Set 1. I think the Set 2 levels maybe have less water generally.
4	anonymous	A and C of set 1 are similar, B of set 2 is similar, set 1 has more similar so set 1
5	anonymous	Idk it just looks like that. I'm going on pure instinct tbh.
6	anonymous	these look pretty similar but set 2 a dosent look the same as single level

Figure 4-32 Why / Why Not? for Question 9 of Section 2 of the Survey

Very split between the correct answer and being unsure, with one participant “going on pure instinct”. Set 2 is described to be “too broken up”, or having less water generally.

Chapter 5 Discussion

This chapter will critically analyse, discuss, and reflect on the results of the survey, as well as the development of the project.

5.1 – Results Discussion

Both sections of the survey have a purpose that they are trying to fulfil, and when both are considered, allows us to draw a conclusion regarding the research question, being if design constraints can affect map diversity.

5.1.1 – Section 1 Interpretation

For every question in this section, except for the last two, the majority of responses are correctly identifying whether the level matches the set or not, with one question even getting a unanimous vote.

The fact that there is a healthy amount of uncertainty and inaccuracy within these primarily correct observations, is promising, for it implies that the design constraints are noticeable, but not restricting the levels enough to look identical. This follows the hypothesis that the design constraints can leave the level diversity mostly unaffected.

For the remaining 2 questions, it should be noted that despite both single levels being part of the set, they both share the long, thin rivers that stretch across the entire level. This is something that the set of levels does not contain an example of. This combined with those 2 questions having the most uncertainty and the most equal split between correct and incorrect answers, there is a relatively high likelihood that this doesn't disprove the earlier observations but rather displays bad luck or design decisions on the researchers end when putting together the survey.

5.1.2 – Section 2 Interpretation

The second section stands out, because the first 2 sets of levels have a near flawless streak of correct, unanimous votes. This may either imply that level diversity is heavily affected, or that the rulesets are easily distinguishable from each other.

When looking at the first pair of sets, one is “LeftRock” and the other “NoOuterEdge”. The trends from these two sets are vastly different, with “NoOuterEdge” leaving many thin strips of water and 1x1 holes, while “LeftRock” has a very similar look as an unconstrained level, other than the rocks only being placed on the side. This leaves a big difference between the amount of water between the sets, making it easy to pair the single level with one of the sets at a glance.

The second pair of sets face a similar issue, with one set being unconstrained levels, and the other being “NoInnerEdge”. These also have drastic differences between them, notably with the amount of water, and the fact that “NoInnerEdge” has many disconnected 1x1 islands and has a general trend to run rivers in between sections of land.

The third pair of sets is “LeftRock”, and unconstrained levels. As mentioned earlier, these 2 rulesets look very similar to each other, which explains why the answers for these 3 questions are so uncertain and mixed. The only noticeable difference is the lack of rocks throughout the level, but this can be easily missed when one is unfamiliar with the rulesets that exist, which all the participants are. It is also important to note that most rulesets affect the water and considering that these are the last questions to be asked in the survey, most participants are likely to be looking at the water, rather than the land.

Considering that all the pairs of sets found in this section are either too distinct, or too similar, it makes it difficult to draw any meaningful conclusion on the data gathered in the section. This does, however, show

that the sample size of rulesets was lacking. Since the rulesets were restricted by the absence of enough design constraints to vary, it displays how that restriction affected the amount of data that could be gathered from this section, and potentially from this project as a whole.

5.1.3 – Overall Survey Interpretation

The survey, while flawed, was able to gather enough data that a conclusion can be drawn. The hypothesis claims that design constraints will not affect level diversity while the number of constraints is kept to a reasonable level. It is unfortunately not possible to test how many constraints need to be active for it to be considered unreasonable, due to the lack of design constraints added as a result of time constraints for the project.

How the project addresses the rest of the hypothesis is possible, however, with the data gathered from the survey. Section 1 makes a very strong argument that the level of design constraints used in this project allows for level diversity to be minimally affected, while still making it clear that the trends are acknowledged. Section 2 is able to back this up, but not with a strong justification, due to the rulesets being so drastically different that with the majority of the questions, there was a unanimous vote on the correct answer.

Overall, the hypothesis was not able to be fully realised, but for the parts that were, it is safe to say that it was correct.

5.2 – Project Discussion

The project as a whole can be seen as a success, but despite this, there are a couple flaws that need to be addressed

5.2.1 – Scope and Motivation

At the beginning of development of this project, it was decided that the algorithm would have 2-4 types of design constraints implemented, to allow for a varied number of different levels to put into the survey. This, however, was not able to be done, and the project ended with one fully fledged design constraint, as well as a simpler “cell weight” attribute. The reason for this is heavily suspected to be burnout and motivation issues for the researcher, as the majority of their life has been spent in education with no long-term break from studies. It is believed that this final semester is the last straw on the camel’s back, and motivation came to a crushing halt. Despite of this, progress has been made on the project, just not to the extent that was hoped.

5.2.2 – Flawed Survey Design

The design of the survey, especially in the second section, was flawed due to each of the rulesets being either drastically different, or near identical. This can be partially excused by the fact that there was a distinct lack of design constraints that could be used for generating the levels, but part of the blame also falls on the researcher, for not considering this possibility during the making of the survey.

Chapter 6 Conclusion and Future Work

To conclude this dissertation, the project as a whole can be seen as a success. The hypothesis, that design constraints does not compromise the map diversity, was proven correct. With the caveat, however, that due to the lack of design constraints able to be implemented because of burnout, an “unreasonable amount” of design constraints was not found. Regardless, with the number of constraints implemented, a “reasonable amount” was not exceeded.

The project artefact functions as intended, and the complex mechanisms found within the code is something that the researcher takes pride in. Considering that all methods of implementing Wave Function Collapse seen by the researcher, use another approach, there is a good possibility that the “Cell and Port” method of implementing WFC is a foreign concept to the internet. This means that, if found to be more efficient to implement or have as a reusable tool, a genuine stride has been made in the field of Wave Function Collapse. This would arguably be considered a greater intellectual advancement than the rest of the dissertation. More work needs to be done.

List of References

Mojang (2009) *Minecraft* [Video Game]. Available at:

<https://www.minecraft.net>.

Plausible Concept (2018) *Bad North* [Video Game]. Available at:

<https://www.badnorth.com>.

Oskar Stålberg (2021) *Townscaper* [Video Game]. Available at:

<https://www.townscapergame.com>.

Sandhu, Chen and Mckoy (2019) *Enhancing Wave Function Collapse with Design-level Constraints*. Available at: [10.1145/3337722.3337752](https://doi.org/10.1145/3337722.3337752)

(Accessed: 5th of May 2026).

BUas Games (2018) *EPC2018 | Wave Function Collapse in Bad North |*

Oskar Stalberg. Available at [https://www.youtube.com/watch?v=0bcZb-](https://www.youtube.com/watch?v=0bcZb-SsnrA)

[SsnrA](https://www.youtube.com/watch?v=0bcZb-SsnrA) (Accessed: 4th of May, 2026).

Ivengar S. and Lepper M. (2000) *When Choice is Demotivating: Can One Desire Too Much of a Good Thing?*. Available at: [10.1037/0022-](https://doi.org/10.1037/0022-3514.79.6.995)

[3514.79.6.995](https://doi.org/10.1037/0022-3514.79.6.995) (Accessed: 4th of May 2026).

Heaton R. (2018) 'The Wavefunction Collapse Algorithm explained very clearly', *Robert Heaton*, 17th of December. Available at:

<https://robertheaton.com/2018/12/17/wavefunction-collapse-algorithm/>.

Appendices

Appendix A – Survey Questions

The survey starts with this text:

You will be shown various levels generated with Wave Function Collapse, and asked questions about how well they fit together. Each section will have the same set of questions for multiple groups of images.

Section 1 starts with this text:

Does Level fit with Set

A set of levels will be presented, and there are rules that determine how all the levels in that set behave and are generated. You will also be presented a single level and asked if that level follows the same rules (if it "belongs") as the level set.

The questions in Section 1 of the survey are as follows:

Does the level on left the follow the same rules as the set of levels on the right?

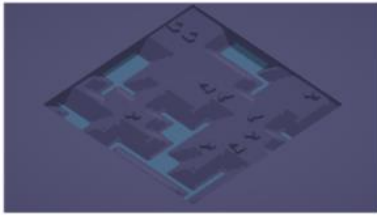
- Belongs with the set
- Unsure
- Does not belong to the set

Why / Why not?

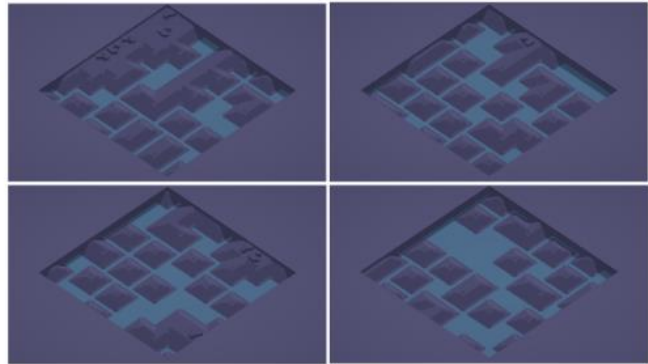
.....

These questions are asked for every image in the first Section.
Below are all images used in Section 1:

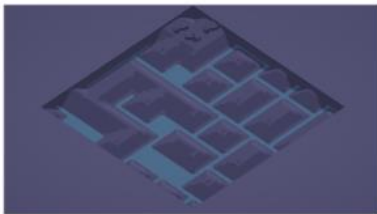
Single Level



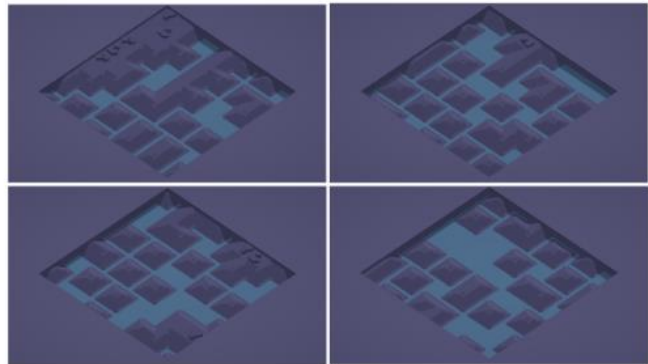
Set of Levels



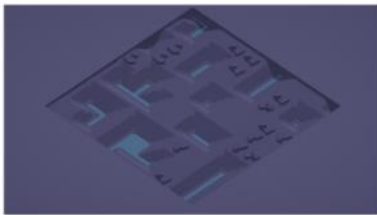
Single Level



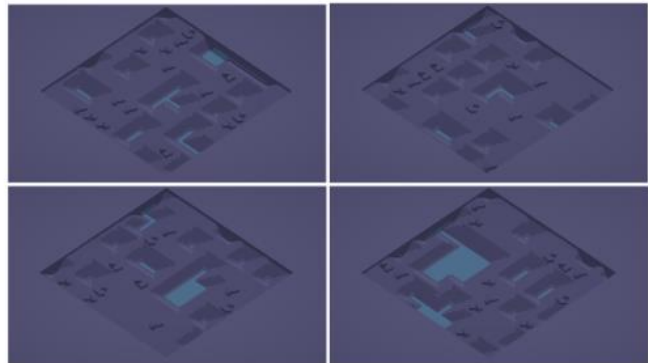
Set of Levels



Single Level



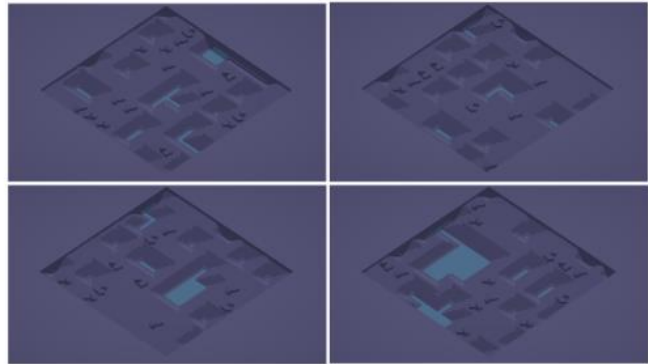
Set of Levels



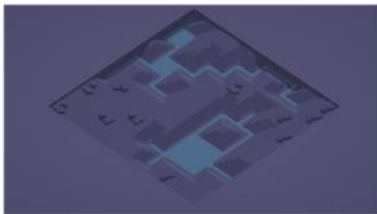
Single Level



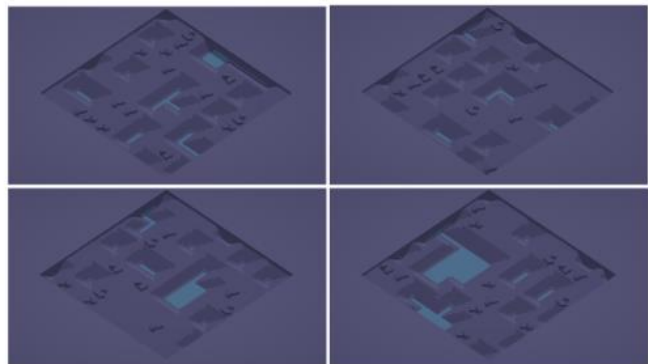
Set of Levels



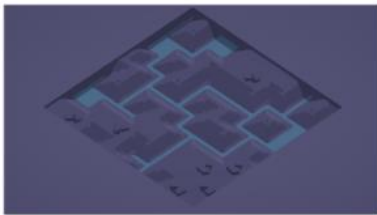
Single Level



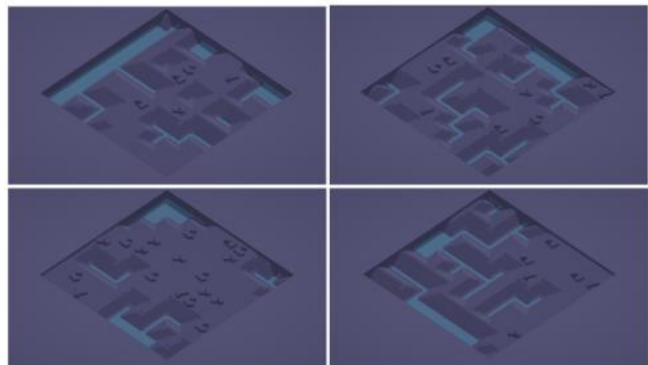
Set of Levels

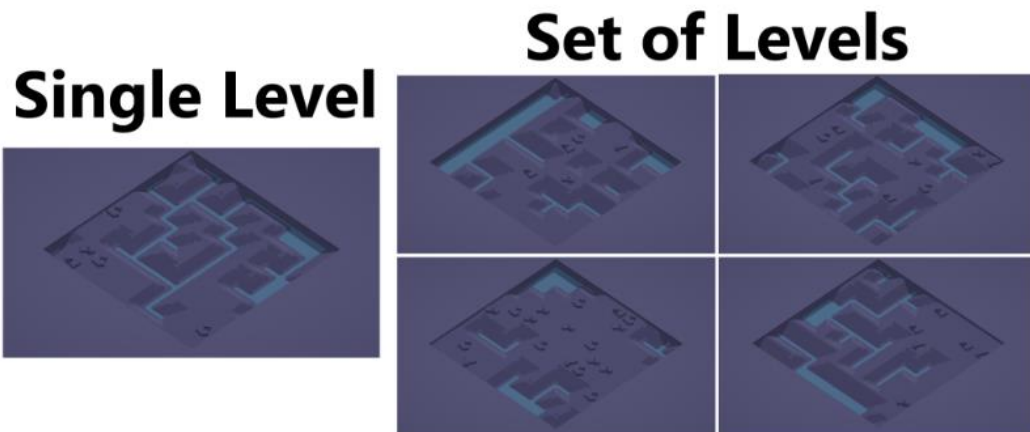


Single Level



Set of Levels





Section 2 starts with this text:

Which set does the level fit with the most

Now, you will be given one single level, as well as 2 separate sets. The single level will belong with one of the two given sets.

The questions in Section 2 of the survey are as follows:

Which set of levels does the level on the left belong to?

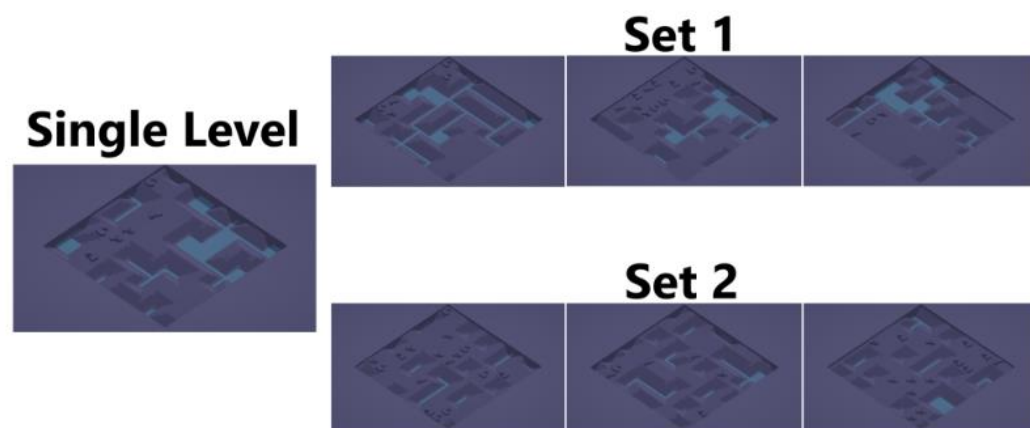
- Set 1
- Set 2
- Unsure

Why / Why not?

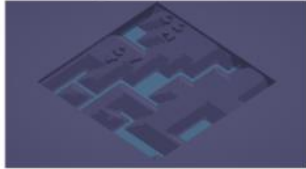
.....

These questions are asked for every image in the second Section.

Below are all images used in Section 2:



Single Level



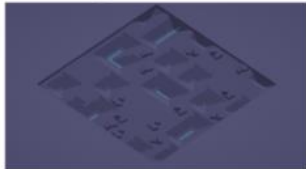
Set 1



Set 2



Single Level



Set 1



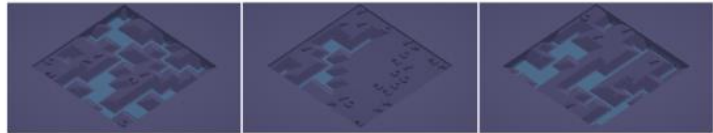
Set 2



Single Level



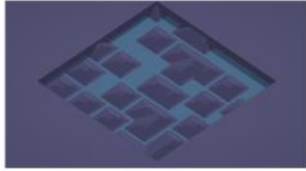
Set 1



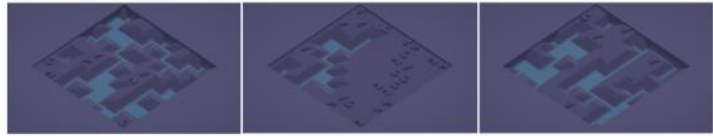
Set 2



Single Level



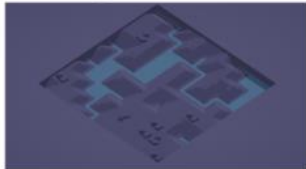
Set 1



Set 2



Single Level



Set 1



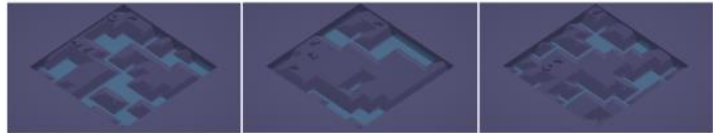
Set 2



Single Level



Set 1



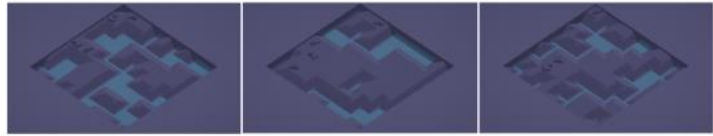
Set 2



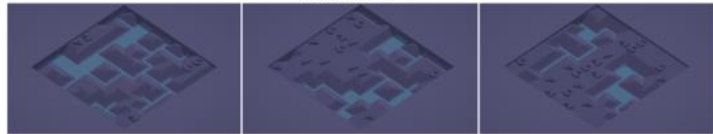
Single Level



Set 1



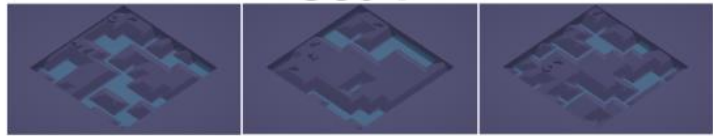
Set 2



Single Level



Set 1



Set 2

